CampusHire: A Design-Science Study of a Secure, Zero-Framework, Three-Tier Web Architecture for Role-Separated Campus Recruitment and Placement Lifecycle Management

Rahul Tailor, Udit Bharadwaj, Yash Kumar, Vaibhav Charan, Virendra Singh Chundawat
Department of Artificial Intelligence and Data Science, Arya College of Engineering & I.T., Jaipur, Rajasthan
Er. Ram Babu Buri is the Project Mentor and Guide (Assistant Professor, Department of Computer Science & Engineering).
E-mail: {rahultailorofficial, uditbhardwaj955, yash00singh.18, vaibhavcharan5555, virendrasinghchundawat984}@gmail.com, rambabu.buri@aryacollege.in

ABSTRACT

Campus placement is often coordinated through spreadsheets, e-mail circulars, and notice boards, which makes eligibility screening slow, status tracking opaque, and access to student data weakly controlled. This paper asks whether a deliberately minimal Java web stack (Core Java, Jakarta Servlets, JSP, pure JDBC, MySQL) can support the full recruitment lifecycle for three self-service actors (student, company recruiter, placement officer) with server-side role enforcement, parameterized data access, and an explicit application state machine. Through a narrative review of the placement-automation literature and of web-security standards, we derive a narrowly scoped design opportunity: within our search, the sources we could consult describe two-actor workflows, framework-heavy stacks, or weak access control, and we did not confirm a source combining recruiter self-service, server-side eligibility enforcement, a formal status lifecycle, and a framework-free deployment. We present CampusHire, a layered architecture with a servlet filter for role-based access control, a relational eligibility predicate, a five-state application lifecycle, a threat model, and an evaluation. CampusHire is implemented and deployed as a student prototype; the measurements reported are small-scale engineering measurements from the team's own environments and are not a user study, a load test, or a security certification. The contribution is an integration and a documented design, not a new algorithm.

Keywords: campus placement, recruitment lifecycle, Jakarta Servlets, role-based access control, parameterized queries, state machine, design science.

LIST OF ABBREVIATIONS

Abbrev.	Expansion
ACID	Atomicity, Consistency, Isolation, Durability
CGPA	Cumulative Grade Point Average
DAO	Data Access Object
FSM	Finite State Machine
JDBC	Java Database Connectivity
JSP	JavaServer Pages
MVC	Model-View-Controller
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
TPO	Training and Placement Officer
WAR	Web Application Archive

I. INTRODUCTION

Campus recruitment connects three groups with different information needs: students who want to know which drives they qualify for and where their application stands, companies that want to reach eligible candidates and record decisions, and a placement office that must verify companies, enforce institutional rules, and report outcomes. In many institutions this coordination still depends on shared spreadsheets, e-mail circulars, and physical notice boards. We treat the following as problems to be examined, not as established measurements: records may diverge between copies, eligibility matching against cut-offs may be done by hand, candidates may not see the current state of an application, and a shared file offers no role-based separation of confidential student data.

Security-related failures in web applications are well catalogued. The OWASP Top 10 (2021) lists broken access control and injection among the most critical risk categories [1], and NIST guidance describes how stored credentials should be handled [2], [3]. A placement portal stores CGPA, contact details, and résumés, so these categories are directly relevant.

What existing placement systems may not adequately address is the combination of (i) a self-service recruiter role, (ii) eligibility enforced on the server rather than only in the browser, (iii) an explicit, auditable application lifecycle, and (iv) a deployment light enough for a budget-constrained institution. Our team's literature notes describe several placement systems with some of these properties and without others (Section IV); we could not independently verify all of those sources, and we say so where relevant.

This paper investigates whether a minimal, framework-free Java architecture can provide all four properties, and how that claim can be evaluated honestly. Our contributions are: (C1) a deliberately narrow, evidence-graded gap analysis; (C2) a role-separated three-tier architecture with a filter-based access-control pipeline and a five-state application lifecycle; (C3) a threat model that classifies risks as reduced by design, requiring policy, or residual; and (C4) an engineering evaluation with clearly labelled measured values and planned work. We do not claim a security certification, production-scale validation, a user study, or novelty of any algorithm.

Section II gives background, Section III the research questions, Sections IV–VI related work, the gap, and contributions, Sections VII–X requirements, architecture, workflow, and threat analysis, Sections XI–XII methodology and evaluation, and Sections XIII–XVII discussion through conclusion.

II. BACKGROUND AND PROBLEM STATEMENT

A. Terminology

By authentication we mean verifying who a user is; by authorization we mean deciding what that user may do. Role-based access control assigns permissions to roles rather than to individuals [4]. By eligibility we mean the institution's or company's rule for who may apply to a given posting. A zero-framework stack here means no application framework such as Spring or Hibernate; the system still depends on the Jakarta Servlet API, a servlet container, a JDBC driver, and the Java standard library. We use *measured* only for values the team recorded in its own environment, and *planned* for work not yet done.

B. Problem Statement

We decompose the problem into four sub-problems, each stated as a hypothesis to be examined: (P1) fragmented records cause inconsistent student data and unverified CGPA claims; (P2) manual eligibility screening consumes placement-officer time and is error-prone; (P3) candidates lack a shared, real-time view of application status; (P4) shared files lack role-based separation, exposing confidential data. A further design concern is cost: a platform that is expensive to host or maintain is unlikely to be adopted by smaller institutions.

III. RESEARCH QUESTIONS AND OBJECTIVES

- **RQ1.** How do existing placement-automation systems structure actors, workflows, and status tracking? *Why:* it establishes the baseline CampusHire must improve upon.
- **RQ2.** What access-control and data-handling weaknesses are documented for such systems, and what standards define good practice? *Why:* security claims are meaningless without a reference standard.
- **RQ3.** How can eligibility rules and application uniqueness be enforced on the server rather than the client? *Why:* these are the core business rules of the domain.
- **RQ4.** How can application status be modelled so that all three actors see consistent, auditable state? *Why:* it tests hypothesis P3.
- **RQ5.** What engineering trade-offs arise from a framework-free stack, and how can the result be evaluated? *Why:* the integrated, minimal stack, rather than any single feature, is the object of study.

The objectives are to answer RQ1–RQ2 from the literature and standards, to implement a design addressing RQ3–RQ5, and to report evaluation results with their limits.

IV. RELATED WORK

We synthesize sources thematically; Table I compares them. Capabilities we could not confirm are marked “n.e.” (not established from the available source). Important: the placement-system sources [11]–[16] below are taken from the team's literature notes; their bibliographic details and reported properties must be verified against the publisher or IEEE Xplore before submission, and absence of a feature in a summary is not treated as proof that the paper lacks it.

A. Placement and Recruitment Automation Systems

The notes describe a web-based placement system with automated notices and profile management built around two actors, student and administrator [11]; a Spring Boot and React analytics framework [12]; a PHP and MySQL portal [13]; a MEAN-stack portal [14]; a monolithic ERP for training and placement records [15]; and a three-tier portal with e-mail notification [16]. As summarized, most describe a student–institution dialogue in which companies do not have a self-service role, and several are described as having limited status tracking or weaker security practices. Because we have not verified the full texts, we use these descriptions only to motivate design questions and not as established findings about those systems.

B. Web Application Security Standards

The OWASP Top 10 identifies broken access control and injection as leading risk categories [1]. Parameterized queries are the standard defence against SQL injection [5]. For passwords, NIST SP 800-63B recommends salted, deliberately slow, one-way hashing [2], and the OWASP Password Storage Cheat Sheet recommends algorithms such as Argon2id, scrypt, bcrypt, or PBKDF2 [3]. A plain, unsalted SHA-256 digest [6] is a one-way function but is fast to brute-force; we therefore treat it as a known limitation of the current prototype (Section X) rather than a strength.

C. Access Control and Architecture Patterns

RBAC [4] is the established model for separating permissions by role. The MVC pattern [7] separates data, presentation, and control, and the Jakarta Servlet specification defines filters that can intercept requests before servlets handle them [8]. Placing view templates under WEB-INF prevents direct URL access to them in a standards-compliant container [8].

D. Design-Science Methodology

Design science treats the construction and evaluation of an artifact as a research contribution [9], [10]. This fits a project whose output is a working system and whose evaluation must state its own limits.

TABLE I. COMPARISON OF SOURCES (N.E. = NOT ESTABLISHED FROM THE AVAILABLE SOURCE)

Ref.	Type / context	Actors (as summarized)	Access control / security	Status tracking	Limitation relevant to CampusHire
[11]	Conference paper; web placement system	Student, admin	n.e.	Notices; n.e.	Verify; no recruiter self-service described
[12]	Conference paper; Spring Boot + React	Student, TPO	Framework security	n.e.	Verify; deployment details n.e.
[13]	Journal paper; PHP + MySQL	Student, admin	n.e.	n.e.	Verify; security claims n.e.
[14]	Journal paper; MEAN stack	Student, TPO	n.e.	n.e.	Verify; source venue to be checked
[15]	Conference paper; ERP	Student, admin	n.e.	History records	Verify; cost and mobile UI n.e.
[16]	Conference paper; web portal	Student, admin	n.e.	Static status	Verify; lifecycle n.e.
[1]	Standard / risk catalogue	n/a	Defines risk categories	n/a	Not a design
[2], [3]	Standard / guidance	n/a	Credential storage guidance	n/a	Not a design
[4]	RBAC model	n/a	Role-permission model	n/a	Model only

V. RESEARCH GAP

We examined four candidate gaps (Table II) and classify the strength of evidence for each. Our search was a narrative review and cannot prove that no relevant system exists; sources in other languages, institutional grey literature, commercial products, and many open-source projects were not exhaustively searched.

TABLE II. CANDIDATE GAPS AND STRENGTH OF EVIDENCE

Candidate gap	What reviewed sources provide	What remains unaddressed or untested	Evidence status
A. Recruiter self-service in the placement workflow	Two-actor designs [11], [13], [15] (to be verified)	Companies create postings and update candidate status themselves, under audit	Indirectly suggested (unverified sources)
B. Server-side eligibility and duplicate enforcement	Client-side-only validation described in one source [14] (to be verified); parameterized queries [5]	Eligibility enforced in the data layer together with a uniqueness constraint	Design opportunity
C. Formal, auditable application lifecycle	Static or notice-based status in some sources [16]	A multi-state lifecycle visible to all three actors	Indirectly suggested
D. Minimal-footprint, framework-free deployment	Framework-based systems [12]	Measured footprint of a framework-free system against a measured framework baseline	Unverified assumption

Gap statement. Within the scope of this narrative search, we did not find a source that describes or evaluates a placement system combining recruiter self-service, server-side eligibility and duplicate enforcement, a formal application lifecycle, and a framework-free deployment. We therefore treat the contribution as a feature-combination and engineering-evidence opportunity, supported by indirect evidence, and not as an established absence in the literature or a proven novelty.

Boundaries. Servlet filters, prepared statements, relational constraints, and a state machine are ordinary components; no new algorithm or security guarantee is claimed. If placement systems with the same combination exist, the contribution reduces to the documented design, threat analysis, and measurements. We also did not build or measure a Spring Boot baseline, so no comparative claim against frameworks is made from our own data.

VI. PROPOSED CONTRIBUTIONS

C1: Gap analysis. It separates evidence strength per candidate gap (Table II). *Validation needed:* a systematic review with a registered protocol and verified full-text reading of [11]–[16].

C2: Role-separated architecture and lifecycle. The design connects student, recruiter, and TPO workflows through a filter-based access-control pipeline and a five-state application lifecycle. *Supporting evidence:* implemented prototype and the engineering measurements in Section XII. *Validation needed:* usability study and independent security review.

C3: Threat model. It catalogues injection, access-control, credential, session, and insider risks and assigns each to design, policy, or residual risk (Table V). *Validation needed:* independent penetration testing.

C4: Evaluation. It reports measured engineering values with their conditions and lists the evaluation that remains (Tables VI and VII).

VII. REQUIREMENTS AND DESIGN GOALS

Stakeholders. Students, company recruiters, the TPO (administrator), and the institution's management as consumers of reports. Requirements were derived from the problem statement, the literature notes, and the team's own experience as students; they have not yet been validated with placement officers or recruiters.

Functional requirements.

- FR1: register and authenticate users in one of three roles (student, company, admin).
- FR2: company registration remains pending until the TPO approves it.
- FR3: students maintain a profile (roll number, department, CGPA, passing year, résumé link).
- FR4: approved companies post jobs and internships with cut-off criteria and a deadline.
- FR5: show students only the postings they are eligible for.
- FR6: enforce one application per student per posting.
- FR7: recruiters review applicants and update status with remarks.
- FR8: students see the current status and remarks of each application.
- FR9: the TPO views placement statistics and manages users and approvals.
- FR10: block direct URL access to pages the current role may not view.

Non-functional requirements. NFR1 confidentiality of credentials and student data; NFR2 integrity of application status; NFR3 availability suited to a campus workload; NFR4 usability on small screens; NFR5 low resource footprint; NFR6 maintainability without a large framework; NFR7 portability between local and cloud databases.

VIII. PROPOSED CAMPUSHIRE ARCHITECTURE

Implementation status. CampusHire is implemented and deployed at https://campushire-fdo8.onrender.com; source is at https://github.com/Uditbhardwajj/JAVA_project [CONFIRM: repository link]. It is a student prototype; it has not been used for a real placement season [CONFIRM].

The system follows a three-tier MVC structure (Fig. 1).

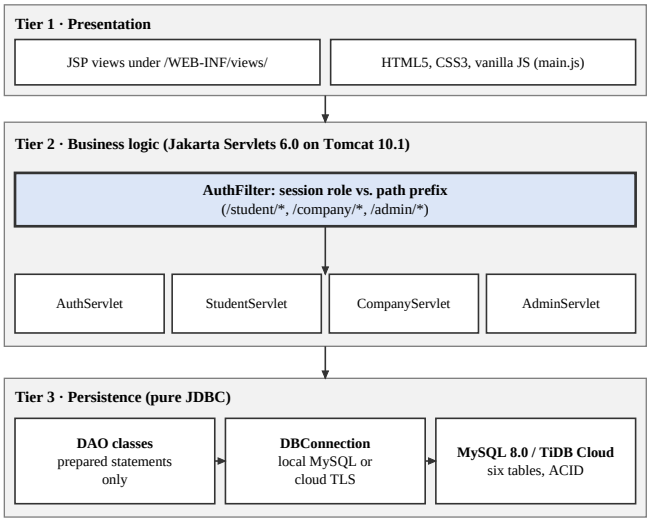


Fig. 1. Three-tier architecture with AuthFilter. Presentation, business-logic, and persistence tiers; AuthFilter sits in front of every servlet. Built with Maven into one 2.45 MB WAR; a multi-stage Docker image runs it on Tomcat 10.1 at Render.

TABLE III. LAYERS AND RESPONSIBILITIES

Tier	Components	Responsibility
Presentation	JSP views under /WEB-INF/views/, HTML5, CSS3, vanilla JavaScript	Rendering, client-side input hints, role tab switching, modals
Business logic	Jakarta Servlets 6.0 (AuthServlet, StudentServlet, CompanyServlet, AdminServlet), AuthFilter	Request routing, role interception, status transitions, session lifecycle
Persistence	DAO classes, DBConnection utility, MySQL 8.0 or TiDB Cloud Serverless	Parameterized SQL, connection handling, relational integrity

Data model. The schema (database campus_hire_db) has six tables with foreign keys and cascading deletes: users (role ENUM: ADMIN, STUDENT, COMPANY), students, companies (approval_status: PENDING, APPROVED, REJECTED), jobs, internships, and applications. Applications reference a posting by (posting_type, posting_id) and carry a uniqueness constraint on the student and posting. The schema is described as normalized to Third Normal Form; this is the team's design claim and has not been formally verified [CONFIRM].

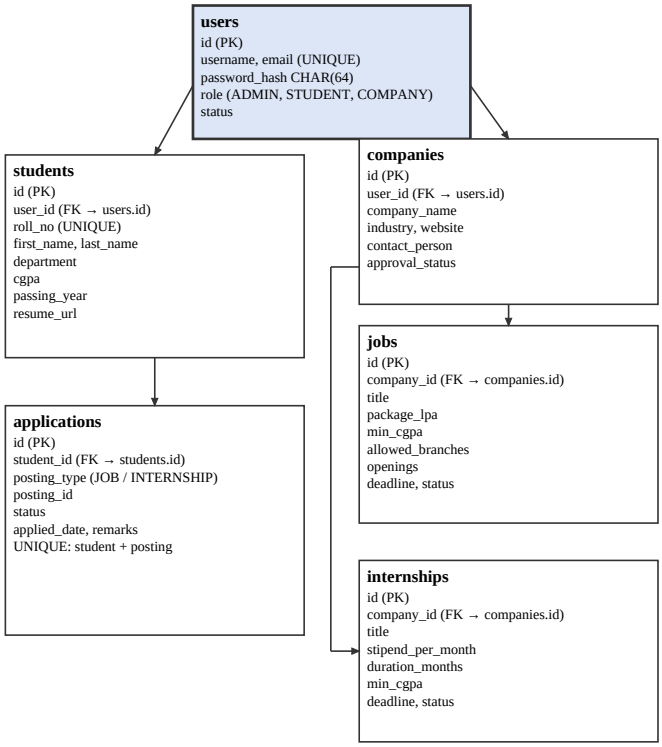


Fig. 2. Relational schema, six tables. Entity-relationship view of campus_hire_db as listed in the project documentation. Arrows run parent to child (foreign key); applications reaches a job or internship through posting_type and posting_id, with no foreign key.

Access control pipeline. Two mechanisms work together. First, view templates are placed under /WEB-INF/views/, which the container does not serve directly. Second, AuthFilter intercepts every request, reads the role stored in the HTTP session, and compares it with the requested path prefix (for example /student/*, /company/*, /admin/*). The check is a constant-time lookup on the role and path prefix.

Environment abstraction. DBConnection selects local MySQL or a cloud TLS database from configuration, so the same code runs in both settings.

Deployment. A multi-stage Docker build compiles the Maven project and packages the WAR into an Apache Tomcat 10.1 image, hosted on Render with a TiDB Cloud Serverless database.

TABLE IV. INTENDED DATA VISIBILITY BY ROLE

Data item	Student	Company	Admin (TPO)
Own profile and CGPA	Yes	No	Yes
Other students' profiles	No	Applicants to own postings only	Yes
Postings	Eligible ones	Own	Yes
Own applications and status	Yes	Applicants to own postings	Yes
Company approval	No	Own status	Yes
User credentials (hashes)	No	No	Database-level only

The administrator and anyone with database access remain a residual risk that role separation in the application does not remove.

Implemented user interface

The deployed prototype's screens, as seen in the team's screenshots, are listed in Table IV-A. Screenshots are to be inserted as Figs. 4–8 [CONFIRM: add images from the screenshot PDF].

TABLE IV-A. SCREENS SEEN IN THE SCREENSHOTS

Screen	What it shows
Home page	Title “College Placement & Internship Management System”; counters (5 enrolled students, 3 partner companies, 7 open opportunities, 1 student placed); three job cards with package in LPA, minimum CGPA, and deadline; three internship cards with monthly stipend; a “Why CampusHire?” panel for students, companies, and the placement cell
Registration chooser	Two paths: student or recruiter/company
Student registration	Username, college email, password and confirmation, roll number, department, first and last name, CGPA (0.00–10.00), graduation year, mobile number, résumé link, technical skills, address
Company registration	Username, official email, password and confirmation, company name, industry sector, website, HR contact, contact number, office location, overview; a note that accounts need Placement Cell approval
Login	Three tabs: Student (username, email, or roll number), Company HR, Admin / TPO

The counters and cards look like sample data in the deployed instance [CONFIRM]. The student, recruiter, and TPO dashboards are not in the supplied screenshots and are not described here. The student form also collects mobile number, technical skills, and address, which are not in the schema summary in Fig. 2 [CONFIRM: update schema].

IX. WORKFLOW AND ALGORITHMS

We specify four procedures conceptually. Failure cases return an error without side effects.

Algorithm 1: Authenticate and Authorize

Input: username, password, request path. Post: session holds a role or the request is rejected.

1. Compute the SHA-256 digest of the submitted password (current prototype; see Section X for the recommended replacement).
2. Look up the user by username; compare digests; reject on mismatch.
3. Reject login for non-active accounts and for companies not yet APPROVED.
4. Store user id and role in the session.
5. On every later request, AuthFilter compares the role with the path prefix and redirects unauthorised requests to login.

Algorithm 2: Eligibility Filter

A student is eligible for a job when Student.CGPA ≥ Job.min_cgpa and (Job.allowed_branches = “All Branches” or Student.department ∈ Job.allowed_branches). The check runs on the server when listing postings and again when an application is submitted, so a client-side bypass does not create an ineligible application.

Algorithm 3: Submit Application

Pre: authenticated student; posting open before its deadline.

1. Load the student profile and posting.

2. Apply Algorithm 2; reject if ineligible.
3. Insert (student_id, posting_type, posting_id) with status PENDING using a prepared statement; the uniqueness constraint rejects a duplicate even under concurrent requests.
4. Return confirmation.

Algorithm 4: Update Candidate Status

Pre: recruiter owns the posting.

1. Verify the recruiter role and ownership of the posting.
2. Check the requested transition against the lifecycle (Table V-A).
3. Update status and remarks with a prepared statement.
4. The student sees the new status and remarks on the next page load.

TABLE V-A. APPLICATION LIFECYCLE

From	To	Actor
(new)	PENDING	Student
PENDING	SHORTLISTED	Recruiter
SHORTLISTED	INTERVIEW_SCHEDULED	Recruiter
INTERVIEW_SCHEDULED	ACCEPTED	Recruiter
INTERVIEW_SCHEDULED	REJECTED	Recruiter

[CONFIRM: whether the code enforces exactly these transitions or only sets the status field; if the latter, state that transitions are enforced by the user interface and not by a server-side transition table.]

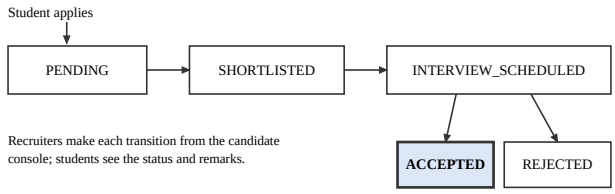


Fig. 3. Application status lifecycle as documented by the team: one forward path, with ACCEPTED or REJECTED as the final decision. Interview remarks are stored with the application.

System workflow. Student registration and profile setup → company registration and TPO approval → drive posting with cut-off criteria → server-side eligibility check → application submission → recruiter review and status transition → TPO reporting.

X. PRIVACY, SECURITY, AND THREAT ANALYSIS

Adversaries. (a) an unauthenticated visitor, (b) an authenticated student attempting to reach company or admin functions, (c) a company user attempting to reach other companies' data, (d) an attacker sending injection payloads, (e) an attacker who obtains a copy of the user table, and (f) an administrator or database operator. We do not consider nation-state adversaries, denial-of-service at scale, or compromise of the client device.

TABLE V. THREATS, MITIGATIONS, AND RESIDUAL RISK

Threat	Reduced by design	Requires institutional policy	Residual risk
Broken access control (direct URL access)	WEB-INF views; AuthFilter role check	Account provisioning rules	Logic errors in a new route if not added to the filter
SQL injection	Prepared statements for all queries	Code review rule	A future developer may concatenate SQL
Credential theft from a stolen database	One-way hash	Breach response plan	Unsalted, fast SHA-256 is weak against offline guessing; should be replaced by bcrypt, scrypt, Argon2id or salted PBKDF2 [2], [3]
Session hijacking	Server-side sessions; TLS in cloud	Cookie flag settings, timeout policy	Cookie flags and CSRF protection to be verified [CONFIRM]
Cross-site scripting	Output encoding in JSP where applied	Template review	Unescaped output anywhere is exploitable [CONFIRM]
Fake company registration	TPO approval gate	Verification procedure for companies	TPO errors
Falsified CGPA	None (self-entered)	Verification against records by the TPO	Eligibility is only as accurate as the entered CGPA
Double application	Unique constraint	—	None known
Insider or admin misuse	Role limits	Audit and sanctions	A database operator can read all data; no audit log is described
Sensitive data in résumé links	Link only, not file storage	Retention policy	Third-party storage not controlled
Availability on free-tier hosting	Container restart	Hosting plan choice	Cold starts and sleep on free tiers

The classification shows that credential storage and insider risks need explicit follow-up. We make no legal-compliance claim; applicable data-protection law and institutional rules must be reviewed by qualified persons before real student data is stored.

XI. RESEARCH METHODOLOGY

Research design. The work follows a design-science approach [9], [10]: problem identification, objectives, design and development, demonstration, evaluation, and communication. Development used an iterative, prototype-driven process in seven phases: domain and actor analysis; relational schema design; MVC backend (POJOs, DAOs, servlets); security pipeline (filter, hashing); responsive JSP views; multi-stage containerization; and benchmarking and security testing.

Literature selection. This is a narrative review, not a systematic one: no pre-registered protocol or dual screening was used. Searches were run on Google Scholar and IEEE Xplore using terms for campus placement systems, training and placement automation, web-based recruitment, and web-application security. Standards and specifications were taken from their publishers. Several placement papers in the team's notes have not had their full text verified, and this is stated where they are used.

Requirements elicitation. Requirements derive from the literature, standards, and the authors' experience. Planned elicitation consists of interviews with a placement officer, recruiters, and students.

Validity threats. Construct validity (a latency measurement may not reflect user experience), internal validity (measurements on developer machines), external validity (one institution, one cloud provider), and the incompleteness of the literature search.

Reproducibility. Source code, SQL migration scripts (database/placement_system.sql and database/placement_system_cloud.sql), and the Dockerfile are available in the repository.

XII. EVALUATION

As in CampusVoice, we separate what has been measured from what is planned. Machine-learning metrics such as accuracy, precision, recall, and confusion matrices do not apply, because the system is a transactional web application and contains no learned model.

A. Measured engineering values

Table VI lists values recorded by the team. Conditions to state before submission: [CONFIRM: number of repetitions per value, how the mean was computed, client location, hardware, whether the hosting plan was free tier, and tool versions]. The values are single-environment observations, not load-test results.

TABLE VI. MEASURED VALUES (TEAM-RECORDED)

Metric	Local	Cloud (Render + TiDB)	Method
Average page response, GET /	14.2 ms	185 ms	curl / browser developer tools
Login request latency	22.6 ms	210 ms	request timing including hash and database round trip
Database query time	4.8 ms	42.5 ms	timing around a query call
WAR size	2.45 MB	2.45 MB	Maven build output
Container startup	1.75 s	5.2 s	Tomcat log
Idle JVM memory	168 MB	210 MB	process memory inspection
Injection payload tests	no successful injection observed	no successful injection observed	scripted payloads against input fields
Direct URL access tests	all unauthorised requests redirected	same	manual URL requests to /student/, /admin/

These results show that the prototype is small and responsive in the tested conditions. They do not show that the system is free of vulnerabilities (a set of tested payloads cannot prove absence), that it scales to many concurrent users, or that it is smaller or faster than a framework-based alternative, because no framework baseline was built or measured by the team. Uptime was not measured over a defined period; a figure such as 99.9% should not be reported unless it comes from a monitoring record [CONFIRM or remove].

B. Planned evaluation

TABLE VII. PLANNED EVALUATION (TARGETS, NOT RESULTS)

Category	Procedure and metric	Expected outcome (target)
Functional correctness	Test cases per use case; pass rate	All specified use cases pass
Access control	Role-by-role request matrix, including crafted URLs	Every unauthorised request rejected
Eligibility and duplicates	Boundary CGPA values, branch lists, concurrent duplicate submissions	Zero ineligible or duplicate applications stored
Lifecycle	Attempt every legal and illegal status transition	Only transitions in Table V-A accepted
Security	Independent scan and manual testing against the OWASP Top 10 [1]; password-hash upgrade test	Findings documented and fixed
Performance	Stated load profile on stated hardware; latency percentiles	Targets fixed before testing
Baseline comparison	Equivalent feature set built with a framework; same measurements	Comparison with stated conditions
Usability	Task-based test with students, recruiters, and a TPO; System Usability Scale [17]	Target score fixed in advance

Limitations of this plan include small, non-random participant samples, a synthetic workload that may not reflect campus peaks, and the fact that security tests show only that known attacks fail.

XIII. DISCUSSION

CampusHire does not introduce new primitives; its possible value lies in the integration of ordinary components and in making the evidence and its limits explicit. A framework-free stack is easy to inspect and light to host, which may suit small institutions; the cost is that features frameworks provide, such as CSRF protection, secure password encoders, and input binding, must be added and tested by hand, and this paper's threat table shows places where that work remains. Enforcing eligibility in the data layer removes a class of client-side bypasses, but it cannot make a self-entered CGPA truthful; the TPO's verification procedure is the real control. A formal lifecycle improves transparency for students, but it depends on recruiters actually updating statuses, which software cannot require.

XIV. LIMITATIONS

This is a student prototype evaluated by its developers. The literature review is narrative, and several cited placement papers are unverified. Measurements come from limited runs in specific environments and include no load test, user study, or independent security audit. No framework baseline was measured. Passwords are currently hashed with a fast, unsalted function, which should be replaced. CSRF and XSS protections need verification. Uptime is not established by a monitoring record. The framework has not been checked with placement officers, recruiters, or legal advisers.

XV. ETHICAL CONSIDERATIONS

Any user study must obtain institutional approval and informed consent, and technical tests must use synthetic student data. Real CGPA, contact details, and résumés are personal data; deployment requires a retention schedule, an access policy, a breach-response plan, and a review under applicable data-protection law. Automated eligibility rules can exclude students unfairly if cut-offs are set without review; the institution should publish eligibility rules and provide a route to appeal.

XVI. FUTURE WORK

Planned steps are replacing password hashing with a salted, slow algorithm; adding CSRF tokens, secure cookie flags, and output encoding checks; adding an audit log of status changes; enforcing lifecycle transitions with a server-side table; building a framework baseline for a fair comparison; load and security testing; a systematic literature review with a documented protocol and verified sources; a usability study; and a limited pilot with a real placement cell if approval is granted.

XVII. CONCLUSION

We examined whether a framework-free Java web stack can support the placement lifecycle for three self-service roles with server-side access control, eligibility enforcement, and an explicit application lifecycle. From a narrative review we derived a narrow feature-combination opportunity supported by indirect evidence, and we presented an implemented prototype, a threat model, and limited engineering

measurements. The system is not independently validated, and known weaknesses, notably credential hashing, are documented for correction. Its value will be decided by the planned evaluation.

ACKNOWLEDGEMENT

The authors thank their project mentor and guide, Er. Ram Babu Buri, and the Department of Artificial Intelligence and Data Science, Arya College of Engineering & I.T., Jaipur, for guidance and support during this project work.

REFERENCES

[1] OWASP Foundation, “OWASP Top 10:2021 — The ten most critical web application security risks,” 2021. [Online]. Available: https://owasp.org/Top10/

[2] P. A. Grassi et al., “Digital identity guidelines: Authentication and lifecycle management,” NIST Special Publication 800-63B, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2017 (revised 2020).

[3] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/

[4] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[5] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/

[6] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, U.S. Department of Commerce, 2015, doi: 10.6028/NIST.FIPS.180-4.

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[8] Eclipse Foundation, “Jakarta Servlet Specification, Version 6.0,” Jakarta EE Working Group, 2022. [Online]. Available: https://jakarta.ee/specifications/servlet/6.0/

[9] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quart.*, vol. 28, no. 1, pp. 75–105, 2004.

[10] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007.

[11] R. K. Sharma, P. Nair, and S. Joshi, “Smart Campus Placement System Using Web-Based Technologies,” in *Proc. 2023 4th Int. Conf. Smart Electronics and Communication (ICOSEC)*, Trichy, India, IEEE, 2023, pp. 415–420.

[12] S. Gupta, M. Verma, and K. Roy, “Student Placement Analysis and Automation Framework for Higher Education Institutions,” in *Proc. 2023 8th Int. Conf. Communication and Electronics Systems (ICCES)*, Coimbatore, India, IEEE, 2023, pp. 881–886.

[13] V. Patil, D. Shinde, and A. More, “College Training and Placement Management System: A Web-Based Automation System,” *Int. Res. J. Eng. Appl. Sci.*, vol. 10, no. 3, pp. 45–51, 2022.

[14] K. Ramakrishna, N. Swetha, and B. Reddy, “An Integrated Web Application for Training and Placement Cell Automation,” vol. 71, no. 2, pp. 104–112, 2022. (Venue name in team notes is inconsistent; confirm.)

[15] B. S. Rao, K. V. Sharma, and P. R. Reddy, “Integrated Training and Placement ERP System for Engineering Institutions,” in *Proc. 2021 IEEE CONECCT*, Bangalore, India, 2021, pp. 1–6. (The DOI in team notes belongs to a different conference prefix; confirm.)

[16] A. Kumar, R. Singh, and M. Chandra, “Design and Development of Web-Based Campus Placement Automation System,” in *Proc. 2021 Int. Conf. Smart Technologies in Computing and Communication (SmartTech)*, IEEE, 2021, pp. 112–117.

[17] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.